



به نام خدا
پوشش دیسک واحد
نام درس
نام و نام خانوادگی



دانشگاه صنعتی امیرکبیر، دانشکده مهندسی کامپیوتر

خردادماه ۱۴۰۱

چکیده

در این گزارش، چهار مورد از الگوریتم‌های تقریبی که اخیراً برای مسئله پوشش دیسک واحد ارائه شده‌اند شرح داده می‌شوند و پس از پیاده‌سازی، با استفاده از چند مجموعه نقطه دنیای واقعی، مورد ارزیابی تجربی قرار می‌گیرند. معیارهای ارزیابی، تعداد دیسک‌های در نظر گرفته شده و زمان اجرای هر الگوریتم خواهد بود. در نهایت عملکرد هر الگوریتم و بهترین الگوریتم‌ها برای هر معیار گزارش می‌شود.

۱ شرح مسئله

مجموعه P شامل n نقطه در صفحه داده می‌شود. هدف، پوشش تمام نقاط با استفاده از کمترین تعداد دیسک با شعاع واحد ($r = 1$) است. این مسئله، پوشش دیسک واحد (UDC)^۲ نامیده شده و یک مسئله ان پی سخت^۱ به حساب می‌آید [۱]. در کاربردهای مختلف مانند شبکه‌های بی‌سیم، تعیین موقعیت، برنامه‌ریزی حرکت، پردازش تصاویر و ... استفاده می‌شود.

۲ مرور الگوریتم‌ها

از سال ۱۹۹۱ تاکنون الگوریتم‌های زیادی ارائه شده‌اند که این مسئله را به صورت تقریبی با فاکتورهای تقریب و پیچیدگی‌های زمانی متفاوت، در نرم اقلیدسی حل می‌کنند. جدول ۱ تاریخچه الگوریتم‌های تقریبی ارائه شده برای این مسئله را نشان می‌دهد.

^۱NP-hard

^۲Unit Disk Cover

مرجع	فاکتور تقریب	پیچیدگی زمانی	سال
[۲]	$2(1 + \frac{1}{l})$	$O(l^2 n^7)$	۱۹۹۱
[۲]	8	$O(n \log S)$	۱۹۹۱
[۳]	$O(1)$	$O(n^3 \log n)$	۱۹۹۵
[۴]	$3(1 + \frac{1}{l})^2$	$O(Kn)$	۲۰۰۱
[۵]	2.8334	$O(n(\log n \log \log n)^2)$	۲۰۰۷
[۶]	$\frac{25}{6}$	$O(n \log n)$	۲۰۱۴
[۷]	4	$O(n \log n)$	۲۰۱۷
[۸]	4	$O(n \log n)$	۲۰۱۸
[۹]	$O(1.321^d)$	—	۲۰۱۸
[۱۰]	7	$O(n)$	۲۰۱۹

جدول ۱: خلاصه الگوریتم‌های تقریبی ارائه شده برای UDC

در ادامه، به ترتیب الگوریتم‌های ذیل بررسی می‌شوند:

- الگوریتم BLMS که در سال ۲۰۱۷ ارائه شده است [۷].
 - الگوریتم LL که در سال ۲۰۱۴ ارائه شده است [۶].
 - الگوریتم DGT که در سال ۲۰۱۸ ارائه شده است [۹].
 - الگوریتم FastCover که در سال ۲۰۱۹ ارائه شده است [۱۰].
- عنوان سه الگوریتم اول، برگرفته از حرف اول نام پژوهشگران مربوط به آن است.

۲۱ الگوریتم BLMS

مجموعه P را به عنوان مجموعه نقاط ورودی که در صفحه قرار دارند و C^* را پوشش دیسک بهینه برای آن در نظر بگیرید. به خاطر داشته باشید که شعاع هر دیسک واحد برابر با ۱ است.

تعریف ۲.۰. در گراف تقاطع دیسک واحد^۳ که با $UDIG(P)$ نمایش داده می‌شود، نقاط موجود در مجموعه P رئوس را تشکیل می‌دهند و برای هر جفت p, q در P یک یال وجود دارد اگر و تنها اگر $|pq| \leq 2$ باشد که $|pq|$ فاصله اقلیدسی بین p و q را نشان می‌دهد.

مشاهده ۲.۱. برای دو نقطه p, q در P ، اگر (p, q) عضوی از $UDIG(P)$ نباشد، آنگاه p و q نمی‌توانند با یک دیسک واحد پوشش داده شوند.

^۳Unit Disk Intersection Graph

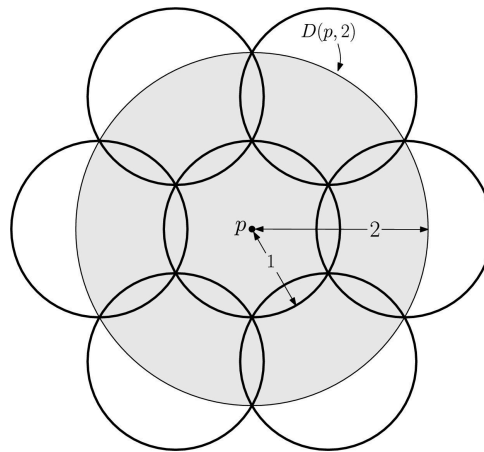
تعریف ۲.۲. یک مجموعه مستقل در $UDIG(P)$ ، زیرمجموعه‌ای مانند I از مجموعه P است، به طوری که هیچ یالی بین جفت $p \in P$

نقطه‌های موجود در I وجود ندارد. همچنین I مجموعه مستقل حداکثری^۴ نامیده می‌شود، اگر برای هر I ، مجموعه $I \cup \{p\}$ در $UDIG(P)$ مستقل نباشد.

فرض کنید I مجموعه مستقل حداکثری در $UDIG(P)$ باشد. طبق مشاهده بالا، اندازه هر مجموعه مستقل در $UDIG(P)$ یک حد پایین^۵ برای تعداد دیسک‌های مورد نیاز به منظور پوشش P است. بنابراین خواهیم داشت:

$$(1) \quad |I| \leq |C^*|$$

مطابق شکل ۱ برای پوشش یک دیسک با شعاع ۲، هفت دیسک واحد با شعاع ۱ لازم و کافی است. بر همین اساس، یک الگوریتم با فاکتور تقریب ۷ برای مسئله UDC به دست می‌آید.



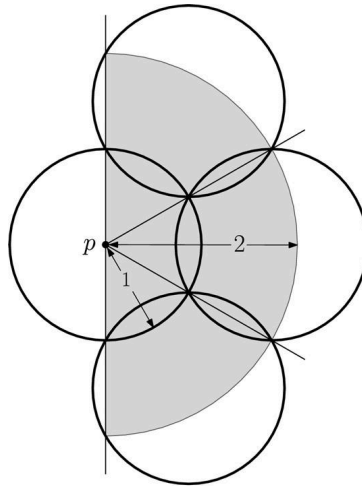
شکل ۱: پوشش $D(p, 2)$ با دیسک‌های واحد

در ادامه خواهید دید که چگونه می‌توان فاکتور تقریب را به ۴ کاهش داد. p را سمت چپ‌ترین نقطه در مجموعه P در نظر بگیرید. در موارد خاص که مقادیر x نقاط با هم برابر است، برای انتخاب سمت چپ‌ترین نقطه، کمتر بودن مقدار y را ملاک قرار می‌دهیم. فرض کنید خط عمودی l از نقطه p عبور کند؛ $R(p)$ اشتراک $D(p, 2)$ با نیم‌صفحه^۶ سمت راست l خواهد بود؛ یعنی $R(p)$ نیم‌دیسک سمت راست $D(p, 2)$ است (مطابق شکل ۲).

^۴Maximal Independent Set

^۵Lower Bound

^۶Half-plane



شکل ۲: پوشش $R(p)$ با دیسک‌های واحد

1. $C = \emptyset$
2. $I = \emptyset$
3. L = List of points in P sorted from left to right
4. while L is not empty:
5. p = first element of L
6. if $d(p, I) > 2$:
7. Cover $R(p)$ by 4 unit disks c_1, c_2, c_3, c_4
8. $C = C \cup \{c_1, c_2, c_3, c_4\}$
9. $I = I \cup \{p\}$
10. $L = L - \{p\}$
11. return C

الگوریتم ۱: نسخه اولیه BLMS

در هر تکرار الگوریتم BLMS، نقطه p به مجموعه I اضافه می‌شود، اگر و تنها اگر $d(p, I) > 2$ باشد. بنابراین p در $UDIG(P)$ به هیچ نقطه‌ای از I متصل نیست. همچنین بعد از خاتمه الگوریتم، I یک مجموعه مستقل حداکثری خواهد بود.

قضیه ۲.۳. فاکتور تقریب نسخه اولیه BLMS برای مسئله پوشش دیسک واحد، ۴ است.

برهان. مجموعه نقاط I و مجموعه دیسک‌های واحد C را در پایان الگوریتم، در نظر بگیرید. براساس نامساوی بالا می‌دانیم که

$|I| \leq |C^*|$ برقرار است. چون برای هر نقطه $p \in I$ ، نیم‌دیسک $R(p)$ با ۴ دیسک واحد پوشش داده می‌شود، مجموعه C

مجموعه P را پوشش می‌دهد. بنابراین رابطه $|C| \leq 4|I| \leq 4|C^*|$ برقرار است. $\square$

پیچیدگی زمانی نسخه اولیه BLMS برابر $O(n \log n + n \cdot t(d))$ است. همچنین محاسبه فاصله با پیچیدگی زمانی

$O(\log^2 n)$ قابل انجام است [۱۱].

1. Initialize event queue Q using ascending x-order of points.
2. Initialize empty BST status T.
3. Initialize empty list C.
4. while Q is not empty:
5. get next event point p and delete it.
6. if p is an end-point: delete corresponding start-point from T.
7. else:
8. find top-2 and bottom-2 neighbors of p in T.
9. if distance to all four neighbors is greater than 2:
10. compute 4 unit-disk centers, append to C.
11. insert p into T.
12. insert $q = (p_x + 2, p_y)$ into Q.
13. return C

الگوریتم ۲: نسخه بهبود یافته BLMS با تکنیک جاروی صفحه

۲۲ الگوریتم LL

در این الگوریتم، صفحه به نوارهای عمودی با عرض $\sqrt{3}$ تقسیم می‌شود. از هر نوار، یک جواب تقریبی با مرتب‌سازی نقاط براساس مؤلفه y به‌صورت نزولی به‌دست می‌آید. نقطه بعدی درون یک نوار که هنوز پوشش داده نشده است، با قرار دادن یک دیسک در پایین‌ترین مکان ممکن، پوشش داده می‌شود. مرکز این دیسک‌ها، روی خطوط عمودی که نوارها را به دو قسمت تقسیم می‌کنند قرار می‌گیرد. جواب نهایی با اجتماع جواب همه نوارها حاصل می‌شود. این سامانه نواری، ۵ مرتبه به سمت راست و هر دفعه به‌اندازه $\frac{\sqrt{3}}{6}$ شیفت داده می‌شود. در هر شیفت، یک جواب به‌دست می‌آید. از بین این ۶ جواب، آن جوابی که کمترین دیسک را استفاده کرده باشد به‌عنوان جواب نهایی در نظر گرفته می‌شود. جزئیات بیشتر در الگوریتم زیر آمده است. فاکتور تقریب این الگوریتم $4.17 \sim \frac{25}{6}$ است.

1. Disk-Centers $\leftarrow \emptyset$, min $\leftarrow n+1$
2. Sort P w.r.t x-coordinate in $O(n \log n)$
3. for i in $\{0,1,2,3,4,5\}$:
4. current $\leftarrow 1$, $C \leftarrow \emptyset$, right $\leftarrow P_1x + i*\sqrt{3}/6$
5. while current $\leq n$:
6. index $\leftarrow$ current
7. while $P_{\text{current}_x} < \text{right}$ and current $\leq n$:
8. current $\leftarrow$ current + 1
9. x-of-restriction-line $\leftarrow$ right - $\sqrt{3}/2$
10. segments $\leftarrow \emptyset$
11. for j from index to current-1:
12. $d \leftarrow P_jx - \text{x-of-restriction-line}$
13. $y \leftarrow \sqrt{1-d^2}$
14. create segment s on restriction line and insert into segments
15. sort segments by non-ascending top y
16. greedily stab segments from top to bottom and add stabbing points to C
17. increment right by a multiple of $\sqrt{3}$ such that $P_{\text{current}} - \text{right} \leq \sqrt{3}$
18. if $|C| < \text{min}$:
19. Disk-Centers $\leftarrow C$, min $\leftarrow |C|$
20. return Disk-Centers

الگوریتم ۳: محاسبه موقعیت دیسک‌های واحد با استفاده از LL

۲۳ الگوریتم DGT

این الگوریتم، ساده و برخط است. به‌ازای هر نقطه‌ای که تاکنون تحت پوشش قرار نگرفته است، یک دیسک واحد به مرکز آن نقطه ایجاد می‌شود. فاکتور تقریب آن در صفحه، ۵ است. در فضای با ابعاد d دارای فاکتور تقریب $O(1.321^d)$ است. برای مشاهده توصیف سطح بالا، به الگوریتم زیر مراجعه نمایید.

1. Disk-Centers $\leftarrow \emptyset$
2. For $p \in P$:
3. if distance to nearest point in Disk-Centers is > 1 :
4. Disk-Centers $\leftarrow$ Disk-Centers $\cup \{p\}$
5. Return Disk-Centers

الگوریتم ۴: محاسبه موقعیت دیسک‌های واحد با استفاده از DGT

۲۴ الگوریتم FastCover

در این الگوریتم، از یک شبکه^۶ با مربع‌های به ضلع $\sqrt{2}$ استفاده می‌شود. هر مربع از این شبکه، می‌تواند توسط یک دیسک به شعاع واحد محاط شود. به‌ازای هر نقطه، اگر توسط یکی از دیسک‌هایی که قبلاً قرار گرفته است تحت پوشش باشد، عملی انجام نمی‌شود؛ در غیر این صورت، یک دیسک واحد به مرکز مربعی که آن نقطه درونش قرار گرفته است، ایجاد می‌شود.

^۶Grid

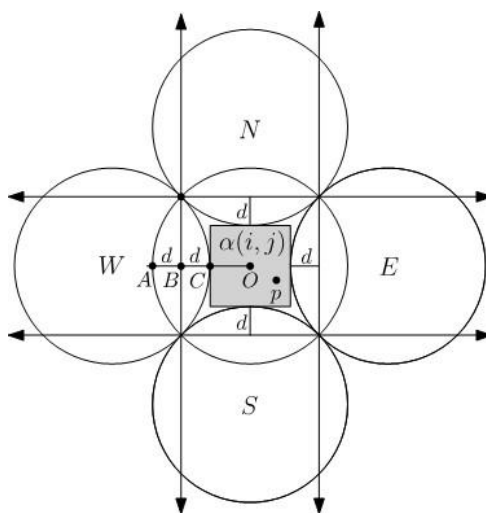
در پیاده‌سازی برای جستجوی سریع‌تر، از یک جدول درهم‌ساز^۸ استفاده می‌شود. در این جدول، مختصات مرکز دیسک‌هایی که اضافه شده‌اند ذخیره می‌شود. برای جلوگیری از مشکلات اعداد اعشاری، از یک جفت عدد صحیح برای نمایش مرکز هر دیسک استفاده می‌شود. مختصات حقیقی می‌تواند با ضرب کردن هر عدد صحیح در $\sqrt{2}$ و اضافه کردن $\frac{1}{\sqrt{2}}$ به آن به دست بیاید. برای به دست آوردن اعداد صحیح از روی یک نقطه، مؤلفه‌های x, y آن بر $\sqrt{2}$ تقسیم می‌شود. این فرآیند در الگوریتم زیر قابل ملاحظه است.

این الگوریتم دارای فاکتور تقریب ۷ و پیچیدگی زمانی $O(n)$ است. یک الگوریتم برخط به حساب می‌آید و هیچ پیش‌پردازشی مثل مرتب‌سازی روی نقاط انجام نمی‌دهد. به اندازه $O(s)$ حافظه اضافی مصرف می‌کند که s نشان‌دهنده اندازه پوشش تولید شده است.

1. $H \leftarrow \emptyset, \text{Disk-Centers} \leftarrow \emptyset$
2. For $p \in P$:
3. $i \leftarrow \text{floor}(p_x/\sqrt{2}), j \leftarrow \text{floor}(p_y/\sqrt{2})$
4. if $(i,j) \notin H$:
5. insert (i,j) into H and corresponding center into Disk-Centers
6. Return Disk-Centers

الگوریتم ۵: محاسبه موقعیت دیسک‌های واحد با استفاده از FastCover

این الگوریتم را می‌توان کمی بهبود داد. وقتی که یک نقطه در یک مربع از شبکه قرار می‌گیرد، ممکن است توسط ۴ دیسک واحد که مربوط به مربع‌های همسایه است و قبلاً اضافه شده‌اند تحت پوشش باشد (مطابق شکل ۳).



شکل ۳: امکان پوشش یک نقطه با دیسک‌های مربع‌های همسایه

^۸Hash table

1. $H \leftarrow \emptyset$, Disk-Centers $\leftarrow \emptyset$
2. For $p \in P$:
3. $i \leftarrow \text{floor}(p_x/\sqrt{2})$, $j \leftarrow \text{floor}(p_y/\sqrt{2})$
4. if (i,j) in H :
5. continue
6. else if east/west/north/south neighbor grid-disk already exists and covers p :
7. continue
8. else:
9. insert (i,j) into H and corresponding center into Disk-Centers
10. Return Disk-Centers

الگوریتم ۶: محاسبه موقعیت دیسک‌های واحد با استفاده از ++FastCover

بهبود دیگری می‌توان روی الگوریتم اعمال نمود. اگر نقاط موجود در دو دیسک مجاور را بتوان با یک دیسک پوشش داد، یعنی قطر مجموعه نقاط هر دو دیسک کمتر از ۲ باشد، می‌توان آن دو دیسک را ادغام نمود. برای ادغام، یک دیسک واحد به مرکز وسط قطر مجموعه نقاط در نظر گرفته می‌شود. برای اینکه محاسبه قطر، تأثیری روی پیچیدگی زمانی الگوریتم نداشته باشد، به‌همراه هر دیسک واحدی که ایجاد می‌شود، یک مستطیل مرزی نیز برای مجموعه نقاط موجود در آن نگهداری می‌شود. هر دفعه که یک نقطه جدید تحت پوشش یک دیسک واحد قرار می‌گیرد، ابعاد مستطیل مرزی مربوط به آن نیز بروزرسانی می‌شود. این بروزرسانی در $O(1)$ قابل انجام است.

1. $H \leftarrow \emptyset$, Disk-Centers $\leftarrow \emptyset$
2. For $p \in P$:
3. $i \leftarrow \text{floor}(p_x/\sqrt{2})$, $j \leftarrow \text{floor}(p_y/\sqrt{2})$
4. if (i,j) in H : update $B(i,j)$ using p
5. else if one of four neighbor grid-disks covers p : update its bounding box
6. else: insert (i,j) into H and initialize $B(i,j)$
7. While there is an unprocessed grid-disk (i,j) in H :
8. if there is a neighbor (k,l) in H with $|i-k| \leq 1$, $|j-l| \leq 1$ and
9. diagonal-length of $B(i,j) \cup B(k,l) \leq 2$:
10. remove both from H and add center of merged box to Disk-Centers
11. For every remaining (i,j) in H :
12. add corresponding grid center to Disk-Centers
13. Return Disk-Centers

الگوریتم ۷: محاسبه موقعیت دیسک‌های واحد با استفاده از ++FastCover

۳ ارزیابی

همه الگوریتم‌ها با زبان ++C و کتابخانه CGAL پیاده‌سازی شده‌اند. هر کدام از آنها بر روی ۱۰ مجموعه نقطه دنیای واقعی اجرا شده‌اند. جدول ۲ نتایج ارزیابی را نشان می‌دهد. منظور از $LL-1P$ ، اجرای یک مرحله‌ای الگوریتم ۳ به جای شش مرحله است.

+FastCover	+FastCover	FastCover	DGT	BLMS	LL-۱P	LL	
+							
,۹۹۹۸۰	,۹۹۹۹۵	,۹۹۹۹۶	,۹۹۹۹۳	,۹۹۹۹۴	,۹۹۹۹۱	,۹۹۹۸۹	birch۳
۰.۰۸	۰.۰۲	۰.۰۲	۰.۰۷	۰.۰۵	۰.۰۳	۰.۱۸	
,۱۰۰۰۰۰	,۱۰۰۰۰۰	,۱۰۰۰۰۰	,۱۰۰۰۰۰	,۱۰۰۰۰۰	,۱۰۰۰۰۰	,۱۰۰۰۰۰	monalisa
۰.۰۸	۰.۰۲	۰.۰۲	۰.۰۸	۰.۱۱	۰.۰۳	۰.۱۵	
,۱۱۵۴۷۵	,۱۱۵۴۷۵	,۱۱۵۴۷۵	,۱۱۵۴۷۵	,۱۱۵۴۷۵	,۱۱۵۴۷۵	,۱۱۵۴۷۵	usa
۰.۱۰	۰.۰۳	۰.۰۲	۰.۰۹	۰.۱۲	۰.۰۴	۰.۱۷	
۰.۰۱,۱۲۵۷	۰.۰۱,۱۳۷۴	۰.۰۱,۱۴۱۸	۰.۰۱,۱۶۲۶	۰.۱۰,۱۶۹۲	۰.۰۴,۱۱۵۲	۰.۱۹,۱۱۴۷	KDDCU۲D
,۱۶۷۸۱۱	,۱۶۸۲۷۷	,۱۶۸۳۳۳	,۱۶۸۰۶۹	,۱۶۸۰۸۸	,۱۶۸۲۷۱	,۱۶۸۲۵۳	europa
۰.۲۰	۰.۰۴	۰.۰۳	۰.۱۶	۰.۱۹	۰.۰۶	۰.۳۵	
۰.۰۶,۶۲۰	۰.۰۴,۶۳۷	۰.۰۴,۶۶۳	۰.۱۲,۷۸۷	۱.۲۱,۸۴۲	۰.۸۸,۶۲۲	۳.۷۴,۶۲۲	wildfires
۰.۰۷,۶۹۶۷	۰.۰۴,۷۵۷۶	۰.۰۳,۷۸۷۴	,۱۰۹۸۰	۰.۷۹,۹۱۴۵	۰.۵۱,۶۶۸۰	۲.۸۴,۶۶۶۷	world
			۰.۱۵				
۰.۱۰,۲۵	۰.۰۵,۳۱	۰.۰۵,۳۴	۰.۱۳,۳۱	۲.۹۰,۳۲	۳.۰۹,۲۶	۱۳.۸۴,۲۵	nyctaxi
۰.۱۶,۴	۰.۰۶,۴	۰.۰۶,۵	۰.۱۹,۵	۴.۰۳,۵	۴.۷۵,۳	۲۱.۰۶,۳	uber
۰.۴۲,۸۴۷	۰.۲۸,۸۶۰	۰.۲۸,۹۰۱	۰.۷۴,۱۱۲۸	,۱۱۹۳	۹.۸۲,۸۸۹	۳۹.۸۳,۸۸۸	hail۲۰۱۵
				۱۱.۱۹			

جدول ۲: نتایج ارزیابی الگوریتم‌ها

۴ نتیجه‌گیری

در مجموعه نقطه‌های دنیای واقعی، اگر معیار با اهمیت‌تر برای ارزیابی، تعداد دیسک‌های استفاده شده باشد، الگوریتم LL عملکرد بهتری دارد. هرچند که الگوریتم ++FastCover در برخی موارد، هم از لحاظ تعداد دیسک‌ها و هم زمان اجرا، عملکرد بهتری داشته است.

اگر معیار مهم‌تر، زمان اجرا باشد الگوریتم FastCover عملکرد بهتری داشته است؛ اما چون تفاوت چندانی با نسخه بهبودیافته خود که تعداد دیسک‌های کمتری تولید می‌کند ندارد، استفاده از ++FastCover پیشنهاد می‌شود.

اگر هردو معیار تعداد دیسک‌ها و زمان اجرا با اهمیت باشد، استفاده از الگوریتم FastCover++ توصیه می‌شود؛ چرا که تعادل خوبی بین هردو معیار برقرار می‌کند [۱۲].

Bibliography

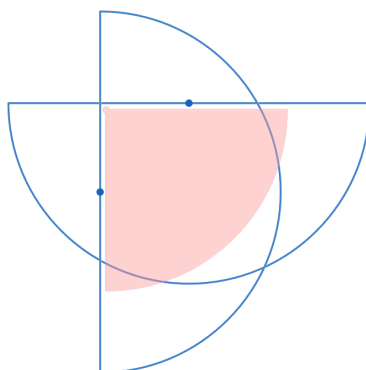
- [١] R. J. Fowler, M. S. Paterson, and S. L. Tanimoto, "Optimal packing and covering in the plane are NP-complete," *Information processing letters*, vol. ١٢, no. ٣, pp. ١٣٣–١٣٧, ١٩٨١.
- [٢] T. F. Gonzalez, "Covering a set of points in multidimensional space," *Information processing letters*, vol. ٤٠, no. ٤, pp. ١٨١–١٨٨, ١٩٩١.
- [٣] H. Brönnimann and M. T. Goodrich, "Almost optimal set covers in finite VC-dimension," *Discrete & Computational Geometry*, vol. ١٤, no. ٤, pp. ٤٦٣–٤٧٩, ١٩٩٥.
- [٤] M. Franceschetti, M. Cook, and J. Bruck, "A geometric theorem for approximate disk covering algorithms," ٢٠٠١.
- [٥] B. Fu, Z. Chen, and M. Abdelguerfi, "An almost linear time 2.8334 -approximation algorithm for the disc covering problem," in *International Conference on Algorithmic Applications in Management*, ٢٠٠٧, pp. ٣١٧–٣٢٤.
- [٦] P. Liu and D. Lu, "A fast $2.5/6$ -approximation for the minimum unit disk cover problem," *arXiv*, ٢٠١٤.
- [٧] A. Biniarz, P. Liu, A. Maheshwari, and M. Smid, "Approximation algorithms for the unit disk cover problem in $2D$ and $3D$," *Computational Geometry*, vol. ٦٠, pp. ٨–١٨, ٢٠١٧.
- [٨] M. Imanparast and S. N. Hashemi, "A simple greedy approximation algorithm for the unit disk cover problem," *AUT Journal of Mathematics and Computing*, vol. ١, no. ١, pp. ٤٧–٥٥, ٢٠٢٠.
- [٩] A. Dumitrescu, A. Ghosh, and C. D. Tóth, "Online unit covering in Euclidean space," *Theoretical Computer Science*, vol. ٨٠٩, pp. ٢١٨–٢٣٠, ٢٠٢٠.
- [١٠] A. Ghosh, B. Hicks, and R. Shevchenko, "Unit disk cover for massive point sets," in *International Symposium on Experimental Algorithms*, ٢٠١٩, pp. ١٤٢–١٥٧.

- [11] J. L. Bentley and J. B. Saxe, “Decomposable searching problems I. Static-to-dynamic transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [12] R. Friederich, M. Graham, A. Ghosh, B. Hicks, and R. Shevchenko, “Experiments with Unit Disk Cover Algorithms for Covering Massive Pointsets,” *arXiv*, 2022.

پیوست

همان‌طور که در جدول ۱ ملاحظه شد، بهترین الگوریتم‌هایی که تاکنون برای مسئله UDC ارائه شده‌اند، دارای فاکتور تقریب ۴ هستند. اینجانب، توجه زیادی به این مسئله با هدف بهبود فاکتور تقریب و نگارش مقاله نمودم. ایده‌های مختلفی برای بهبود فاکتور تقریب به ذهنم رسید که پس از بررسی‌های فراوان متوجه شدم برخی از آنها اشتباه است و برخی دیگر را به‌دلیل کمبود زمان نتوانستم به‌طور دقیق بررسی کنم.

ایده اول، با هدف کاهش فاکتور تقریب از ۴ به ۳ بود. در شکل ۲ نشان داده شد که برای پوشش یک نیم‌دیسک به شعاع ۲، دقیقاً به ۴ دیسک واحد احتیاج است. پس از بررسی مشخص شد که این ایده عملی نیست، چون فرض اولیه اشتباه است و حالت‌هایی وجود دارد که اشتراک دو نیم‌دیسک بیشتر از حد تصور می‌شود و نمی‌توان آن را با یک ربع‌دیسک پوشش داد (مطابق شکل ۴).



شکل ۴: عدم امکان پوشش اشتراک دو نیم‌دیسک با ربع‌دیسک

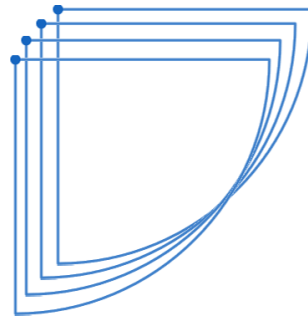
ایده دوم نیز با هدف کاهش فاکتور تقریب از ۴ به ۳ بود. به‌جای نیم‌دیسک‌های با شعاع ۲، ربع‌دیسک‌هایی به شعاع ۲ در نظر گرفته می‌شود که هرکدام با ۳ دیسک واحد قابل پوشش هستند.

1. Initialize event queue Q in descending y -order.
2. Initialize empty BST T and list C .
3. while Q is not empty:
4. process start/end events of quarter-disks.
5. if left neighbors are farther than 2:
6. add 3 covering unit disks.
7. return C

الگوریتم ۸: ایده دوم برای کاهش فاکتور تقریب به ۳

پس از بررسی، مشخص شد که این ایده نیز عملی نیست. زیرا مانند شکل ۵ حالت‌هایی وجود دارد که ربع‌دیسک‌های زیادی با

یکدیگر همپوشانی پیدا می‌کنند و در نهایت فاکتور تقریب، $O(n)$ می‌شود.



شکل ۵: حالتی که ربع‌دیسک‌های زیادی همپوشانی پیدا می‌کنند